Task 1 Report

Most of the outputs required for the report are already documented in the code!

Part 1

- **Padding logic:** The padding logic was implemented using the `numpy` library (`np.pad`), which added a constant value of zero around the image.
- **Why the corner is darker than the rest of the image:** The corner appears darker because zero-padding introduces virtual black pixels outside the image boundaries, which pull down the average pixel intensity during convolution.

Part 2

- **Why must the entries sum to 1?** The entries must sum to 1 because we are taking the average of the image.
- **Why the analytic factor $\frac{1}{2\pi\sigma^2}$ is not sufficient:** Because of the truncated area and discrete sampling, summing all values in the unnormalized array yields a total slightly less than 1.0 (e.g., 0.983). If applied directly to an image, this energy deficit causes the entire image to become darker with every convolution step. Dividing every element by the kernel's total sum re-normalizes the sum to exactly 1.0, preserving total image brightness.
- **What changes, and what σ controls:** In practical terms, increasing σ expands the spatial footprint of the Gaussian bell curve, spreading the smoothing weight across a wider neighborhood of pixels. As σ grows, high-frequency details, fine textures, and sharp noise are smoothed away into broader, softer gradients. Ultimately, σ acts as a scale parameter that controls the level of abstraction in the image, determining the minimum feature size that survives the blurring operation.
- **Choosing kernel size (k):** The formula used is $k > 6\sigma + 1$. The parameter k should be greater than σ because otherwise, it fails to catch the wide range of the Gaussian curve, which is almost as mapping all pixels location to the same value.

As σ increases, we take much more area from the image and map it to a single pixel. But as the kernel size increases, the picture gets more blurry.

Part 3

- **Sanity Checks:** The outputs verify the expected math for the DoG kernels, ensuring the array values are properly structured.
- **Edge Gradient Direction:** For the upper left image for example, it is the G_x which is the gradient in the x direction. This makes sense because of how the array is written.
- **Comparison of Steps (2), (3), and (4):** Step (2) applies raw Sobel differentiation directly to a noisy image, which amplifies high-frequency noise and degrades the edge map. Step (3) smooths the noisy image with a Gaussian filter first and then applies the Sobel operator in a separate second step. Step (4) convolves the noisy image directly with a single Derivative of Gaussian (DoG) kernel.

Steps (3) and (4) produce nearly identical output images. This identity occurs because linear 2D convolution is associative and distributive; mathematically, taking the derivative of a smoothed image is identical to convolving the image with a differentiated Gaussian function:

$$\frac{\partial}{\partial x}(I * G) = I * \left(\frac{\partial}{\partial x} G\right)$$

Part 4

- **Filter Choice:** I would say Gaussian kernel, cause it has more freedom for the output image.
- `cv2.Sobel` **Comparison (**`cv2.CV_8U` **vs.** `cv2.CV_64F`**):** The fundamental difference between the two outputs stems from how negative derivative values (gradients) are stored during convolution.
  - **Unsigned 8-bit Integer (**`cv2.CV_8U`**):** Standard images store pixel intensities in the range $[0, 255]$. When a derivative kernel transitions from a light pixel to a dark pixel, the resulting slope is negative. Because `CV_8U` cannot store negative numbers, OpenCV automatically truncates all negative gradient values straight to 0 (black). As a result, `cv2.CV_8U` only detects edges transitioning from dark-to-light, completely missing the opposite side of objects.
  - **64-bit Floating-Point (**`cv2.CV_64F`**):** Using `cv2.CV_64F` preserves signed decimal precision, keeping both positive gradients (dark-to-light transitions) and negative gradients (light-to-dark transitions). Taking the absolute value (`np.abs()`) or scaling the floating-point matrix prior to display ensures that boundaries in both directions are properly captured and visualized.